



**¡Les damos la
bienvenida!**

¿Comenzamos?

Esta clase va a ser

- grabada
a

¿Dudas sobre el onboarding?

Mirá la clase 0

DESAFÍOS Y ENTREGABLES

Proyecto final

El Proyecto final se construye a partir de **los desafíos** que se realizan clase a clase. Se va creando a medida que el estudiante sube los desafíos entregables a nuestra plataforma.

El objetivo es que cada estudiante pueda utilizar su Proyecto final como parte de su portfolio personal.

Se debe subir a la plataforma la ante-última o última clase del curso. En caso de no hacerlo **tendrás 20 días** a partir de la finalización del curso para cargarlo en la plataforma. Pasados esos días el botón de entrega se inhabilitará.



¿Cuál es nuestro
Proyecto final?



PROYECTO FINAL

Backend de una aplicación de e-commerce

Desarrollarás el **backend de una aplicación de e-commerce** para poder vender productos de un rubro a elección.

El servidor se basará en un diseño de capas, orientado a MVC y su código contendrá las estructuras de programación más sólidas por parte del lenguaje ECMAScript.





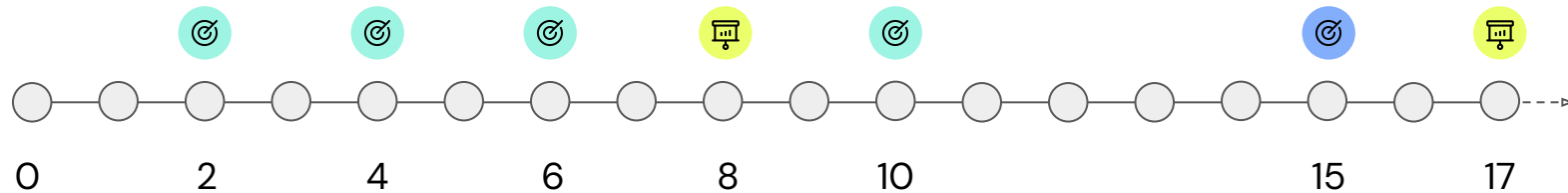
PROYECTO FINAL

Entrega	Requisito	Fecha
Primera pre entrega	Primera integración de tu PF	Clase 8 – M1
Segunda pre entrega	Profesionalizando la BD de tu servidor	Clase 17 – M1
Tercera pre entrega	Mejorando la arquitectura de tu PF	Clase 30 – M2
Entrega de PF	Proyecto Final	Clase 46 – M3

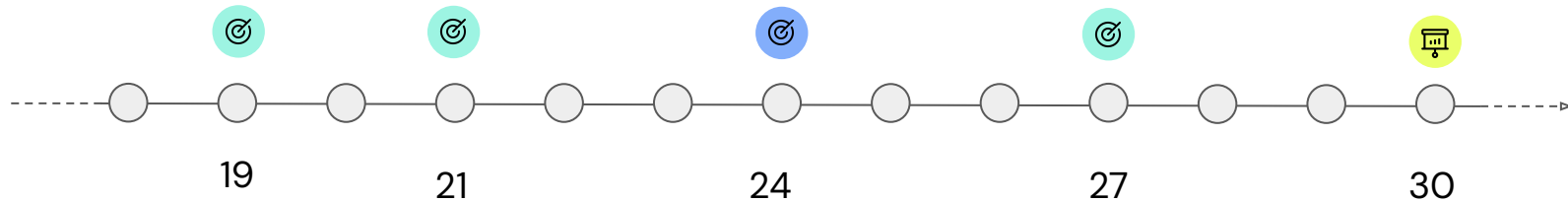


GRILLA DE ENTREGAS

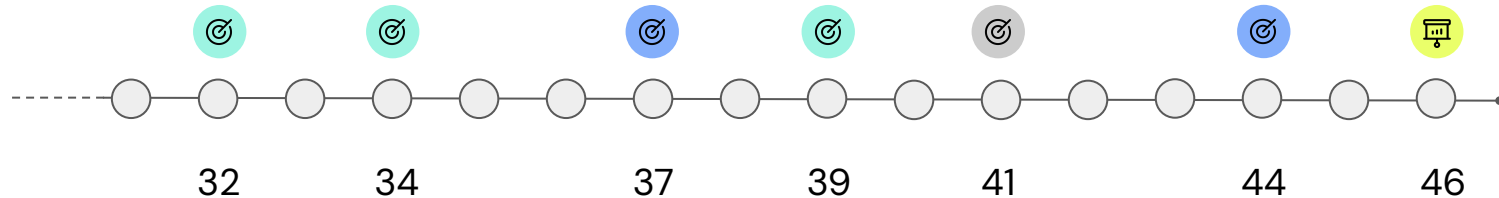
M1



M2



M3



Principios básicos de Javascript

Temario

00

Principios de programación Backend

- ✓ Programación web
- ✓ Distintas maneras de probar Javascript
- ✓ Tipos de datos en Javascript

01

Principios básicos de Javascript

- ✓ [Tipos de datos y variables en Javascript](#)
- ✓ [Javascript y ES6](#)
- ✓ [Funciones en Javascript](#)

02

Funcionalidades de los lenguajes ECMAScript

- ✓ Nuevas funciones de ECMAScript
- ✓ Funciones más utilizadas en backend

Objetivos de la clase



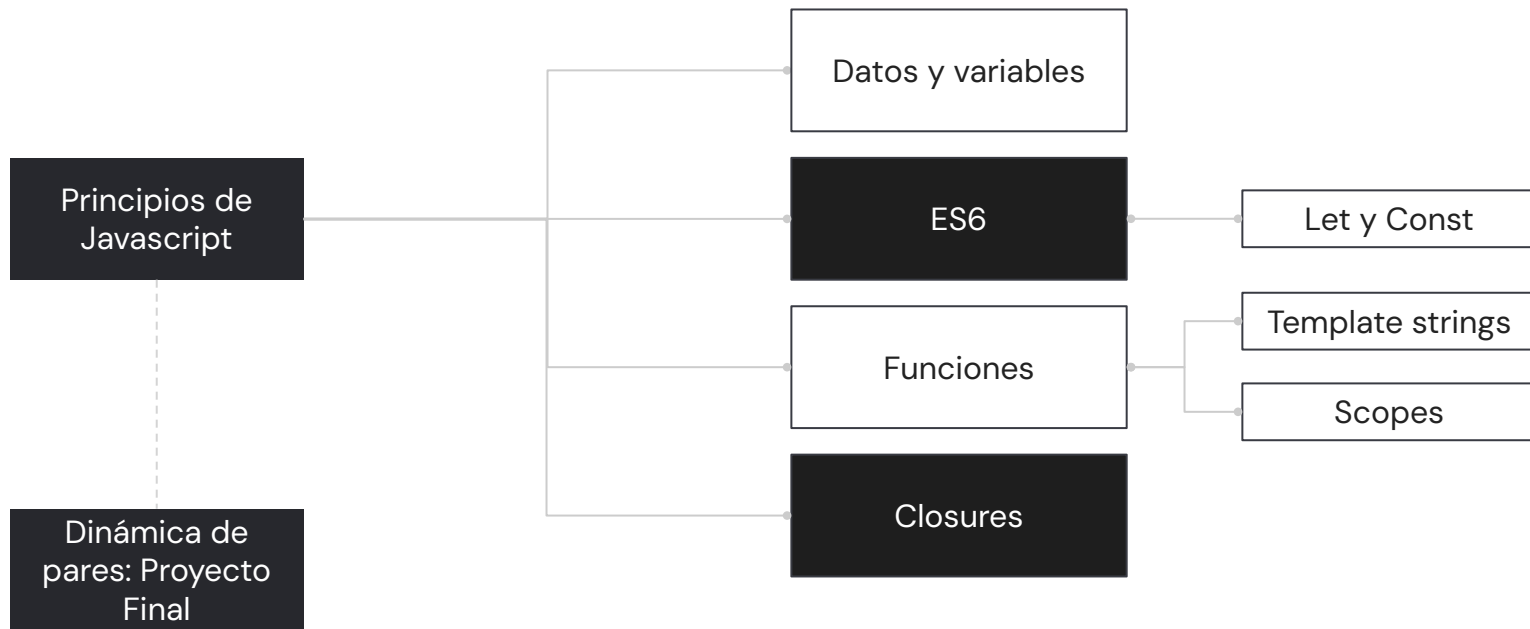
Familiarizarse con las estructuras y conceptos fundamentales al programar utilizando Javascript



Conocer los nuevos elementos de lenguaje aportados por ES6



MAPA DE CONCEPTOS



Tipos de datos y **variables** en Javascript



Repaso

En la clase 0 recordamos los tipos de datos con los que podemos trabajar en Javascript.

¡Pongamos en juego ese conocimiento!

Veamos un bloque de código e intentemos identificar los tipos de datos que están presentes en él.

Duración: **10 minutos**

¿Qué tipos de datos identifican?

JS EjercicioTiposDeDatos.js > ...

```
1  let people = [  
2    {id:1,first_name:"Luz",last_name:"Escalante",age:25,gender:"F"},  
3    {id:2,first_name:"Bruno",last_name:"Guerra",age:18,gender:"M"},  
4    {id:3,first_name:"Marisol",last_name:"Cadena",age:23,gender:"F"},  
5    {id:4,first_name:"Franco",last_name:"Chachagua",age:30,gender:"M"}  
6  ];  
7  
8  let person = people.find(p=>{  
9    let test;  
10   test = p.id===3;  
11   return test;  
12 })  
13  
14 console.log(person);  
15
```

Actividad

		Nombre	Descripción
Tipos de datos en Javascript	Tipos primitivos ¿qué características comparten?	String	Cadenas de texto
		Number	Valores numéricos
		Booleans	True/false
		Null	Tipo especial, contiene null
		Undefined	Tipo especial, contiene undefined.
	Tipos objeto ¿qué características comparten?	Tipos predefinidos de Javascript	Date (fecha) RegExp (expresiones regulares) Error (datos de error)
		Tipos definidos por el programador/usuario	Funciones simples Clases
		Arrays	Serie de elementos o formación tipo vector o matriz. Lo consideraremos un objeto especial que carece de métodos.
		Objetos especiales	Objeto global
			Objeto prototipo
			Otros



Para pensar

Una variable es un **espacio de memoria** apartado por la computadora, para poder **guardar un dato**.

¿Verdadero o falso?

Contesta mediante el chat de Zoom



Para pensar

Una variable **no puede cambiar su valor**, a pesar de que el programa lo necesite. Por consiguiente, no puede reutilizarse.

¿Verdadero o falso?

Contesta mediante el chat de Zoom

Javascript y ES6

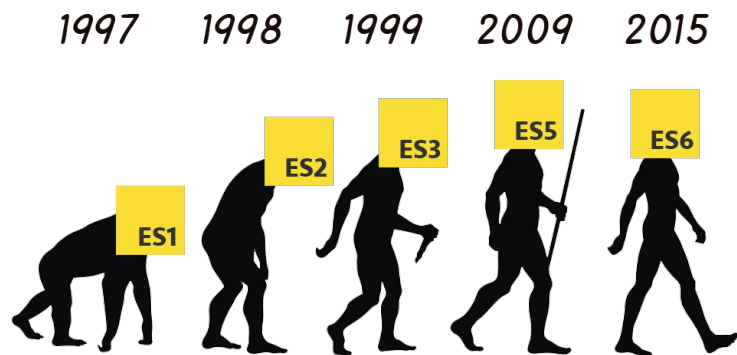
¿Qué es ECMAScript?

Es un estándar que se encarga de regir **cómo debe ser interpretado** el lenguaje de javascript. Con el avance del lenguaje y el crecimiento de los navegadores era necesario establecer un control.

ECMAScript plantea compatibilidades, nuevas funciones, funciones en desuso y demás en cada uno de sus nuevos releases.

Cada nuevo release (versión) de ECMAScript que sale, se denota por ES+número de release:

ES6, ES7, ... , ES11



Algunos cambios importantes de ES6

Let y const

`let` y `const` son dos formas de declarar variables en JavaScript introducidas en ES6 que limitan el ámbito de la variable al bloque en que fue declarada (antes de ES6 esto no era así).

Es posible que se encuentren con ejemplos y código en internet utilizando la palabra reservada “`var`” para crear variables. Esta es la manera en que se hacía antes de ES6 ¡y no se recomienda su uso!



Let

Un bloque en JavaScript se puede entender como “lo que queda entre dos llaves”, ya sean definiciones de funciones o bloques `if`, `while`, `for` y loops similares.

Si una variable es declarada con `let` en el ámbito global o en el de una función, la variable pertenece al ámbito global o al ámbito de la función respectivamente.

Ejemplo let

Aquí la variable `i` es global y la variable `j` es local.

Pero si declaramos una variable con `let` dentro un bloque, que a su vez está dentro de una función, la variable pertenece solo a ese bloque.

```
let i = 0;
function foo() {
  i = 1;
  let j = 2;
  if(true) {
    console.log(i); // 1
    console.log(j); // 2
  }
}
foo();
```

```
function foo() {
  let i = 0;
  if(true) {
    // Sería otra variable i
    // sólo para el bloque if
    let i = 1;
    console.log(i); // 1
  }
  console.log(i); // 0
}
foo();
```


Ejemplo let

```
function foo() {  
  if(true) {  
    let i = 1;  
  }  
  // ReferenceError:  
  // i is not defined  
  console.log(i);  
}  
foo();
```

Fuera del bloque donde se declara con `let`, la variable no está definida.

Const

Al igual que con `let`, el ámbito (scope) para una variable declarada con `const` es el bloque.

Sin embargo, `const` además prohíbe la reasignación de valores (`const` viene de `constant`).

Ejemplo const

```
const i = 0;  
  
// TypeError:  
// Assignment to constant variable  
i = 1;
```

Si se intenta reasignar una constante se obtendrá un error.

Mutabilidad y const

- ✓ Mientras que con `let` una variable puede ser reasignada, con `const` no es posible.
- ✓ Si se intenta reasignar una constante se obtendrá un error tipo `TypeError`.
- ✓ Pero que no se puedan reasignar no significa que sean inmutables.
- ✓ Si el valor de una constante es algo "mutable", como un array o un objeto, se pueden cambiar los valores internos de sus elementos.

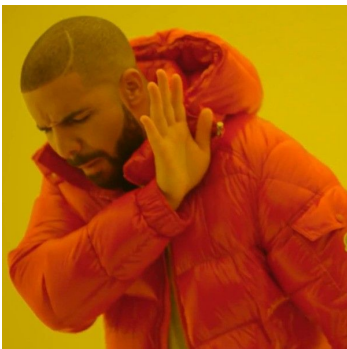
NO REASIGNABLE $\neq$ INMUTABLE



Ejemplo en vivo

- Se declara una constante de tipo string. Intentamos cambiar el valor de dicha string. **¿Es posible?**
- Se declara una constante de tipo Array<number>. Intentamos cambiar el último y el primer elemento. **¿Es posible?**
¿Por qué?

Ejemplo de mutabilidad



```
const user = 'Juan';  
//TypeError: Assignment to constant  
user = 'Manolo';
```



```
const user = { name: 'Juan' };  
user.name = 'Manolo';  
console.log(user.name); // Manolo
```

Funciones en Javascript

¿Qué es una función?

Son bloques de instrucciones que trabajan sobre un scope interno (conocido como scope local). Pueden encontrarse en su sintaxis básica o en su notación flecha.

```
replace_interests => false,  
'send_welcome' => false,  
});  
  
array('error', $result)) {  
    $result = array('response' => 'error', 'mes  
{  
    $result = array('response' => 'success');  
    encode($arrResult);  
}
```




Ejemplo en vivo

- Realizar una función con estructura básica. Se destacarán los elementos y un caso de uso.
- Realizar la misma función con estructura flecha. Se destacarán las diferencias.

Ejemplos de los diferentes tipos de funciones

```
JS 1.js > sumarDosNumeros
1 function nombreDeLaFuncion(parametros){
2     /*Cuerpo de la función, todas las instrucciones
3     * internas que necesitamos que la función
4     * realice.
5     */
6     let variableParaMiFuncion=2;
7     return variableParaMiFuncion;
8     /* Con la palabra "return" podemos MANDAR FUERA
9     * DEL SCOPE alguna variable que necesite en
10    * otra parte del código.
11    */
12 }
13 /*EJEMPLO COMPLETO*/
14 function sumarDosNumeros(numero1,numero2){
15     //resultado solo existe dentro de la función
16     let resultado;
17     resultado = numero1+numero2;
18     /*Cuando la función acabe, "resultado" muere
19     * así que hay que mandarla afuera
20     */
21     return resultado;
22 }
23 //Mandamos llamar a la función con valores reales
24 let total = sumarDosNumeros(2,3);
25 console.log(total) //5
```

```
JS 2.js > identificadorDeFuncion
1 /*Una función flecha es ANÓNIMA, quiere decir que
2 * No tiene nombre, pero puede asignarse a una
3 * variable para poder identificarse.
4 */
5 const identificadorDeFuncion = (parametros) =>{
6     /*Cuerpo de la función, todas las instrucciones
7     * internas que necesitamos que la función
8     * realice.
9     */
10    let variableParaMiFuncion=2;
11    return variableParaMiFuncion;
12    /* Con la palabra "return" podemos MANDAR FUERA
13    * DEL SCOPE alguna variable que necesite en
14    * otra parte del código.
15    * La función flecha cuenta con un return
16    * IMPLÍCITO
17    */
18 }
19 /*EJEMPLO COMPLETO */
20 const sumarDosNumeros = (numero1,numero2) =>{
21     let resultado;
22     resultado = numero1+numero2;
23     return resultado;
24 }
25 const sumarReturnImplicito = (num1,num2)=>num1+num2;
```



PARA RECORDAR

Contenido destacado

La función flecha permite un return implícito, lo cual permite utilizar instrucciones sin llaves. Esto sólo es posible si la función tiene **una instrucción**. Verás muchas de estas en un ambiente laboral.

Si la función flecha sólo tiene un argumento, no es necesario encerrar el parámetro en un paréntesis. Esto sólo es necesario al utilizar dos argumentos o más.

Scopes

Scopes

El scope define el alcance de una variable o constante a un cierto contexto. Esto permite utilizar el mismo nombre para diferentes variables, sin confundir al computador.

El **scope global** afectará a todo el nivel del archivo en el que trabajemos, mientras que el **scope local** afectará a la función o bloque en el que esté declarado.

Ejemplo de scope no válido

```
function exampleFunction() {  
  // x solo se puede utilizar en exampleFunction  
  const x = 'declarada en el scope local'  
  console.log(x)  
}  
  
console.log(x) // ReferenceError: x is not defined
```

Si la variable está definida exclusivamente **dentro de la función**, no será accesible desde fuera de la misma o desde otras funciones.

Ejemplo de scope válido

```
const x = 'declarada en el scope global'

function example() {
  console.log(x) // x existe acá adentro
}

example() // esto no lanza error

console.log(x) // x existe acá afuera también
```

El siguiente código es válido debido a que la variable se declara **fuera de la función**, lo que la hace global.

Template strings

Template strings

- ✓ Funcionan como un **superset** de una string.
- ✓ Permite **incrustar** información dentro de ella, evitando la concatenación.
- ✓ Reconoce los saltos de línea, para mantener el formato.
- ✓ Pero más importante aún es que no presenta el límite de caracteres de una string normal (Permitiendo hacer estructuras más complejas, como plantillas)

```
`texto de cadena de caracteres`  
  
`línea 1 de la cadena de caracteres  
línea 2 de la cadena de caracteres`  
  
`texto de cadena de caracteres ${expresión} texto adicional`
```



Hands on lab

En esta instancia de la clase **realizaremos una función que corrobore elementos en una lista.**

¿De qué manera?

El profesor demostrará cómo hacerlo y tú lo puedes ir replicando en tu computadora. Si surgen dudas las puedes compartir para resolverlas en conjunto de la mano de los tutores.

Funciones

¿**Cómo lo hacemos?** Definiremos la función “mostrarLista”, la cual recibirá un arreglo con elementos como parámetro.

- ✓ Si la lista está vacía, devolver un mensaje indicando “Lista vacía”.
- ✓ Si la lista cuenta con elementos, mostrarlos 1 por 1 en consola. Finalizar el proceso devolviendo la longitud de la lista (Utilizar template strings)
- ✓ Invocar la función con los casos de prueba.

Se espera una duración de 10 minutos.



Break

¡10 minutos y volvemos!

Closures en Javascript

Closures

Una cláusula o closure es una función que puede mantener variables declaradas de manera interna, además de contar con una función que pueda acceder a ambos ámbitos, tanto el suyo como el de su función padre, logrando así un efecto de **variable privada**.

Este efecto de encapsulamiento se utilizaba con anterioridad debido a la falta de la implementación de **clases** en Javascript, sin embargo, al introducirse las clases, el cambio fue muy notorio, dejando a las closures en desuso, o bien en casos de uso muy limitados e insuficientes para códigos enterprise.

Uso de **clases** en Javascript

Clases

Una clase es una representación de una entidad. Nos sirve para modelar múltiples cosas como: un auto, una persona, o bien cosas más abstractas como: un administrador de archivos, un conector a base de datos, etc.

Las clases funcionan como **moldes**, por lo que una vez definida una de éstas, podemos crear múltiples objetos con la misma forma y con las mismas funcionalidades. A éstas se les llaman **instancias**.

Declaración de clases

Ejemplo de una estructura general de clases, con métodos y variables

```
JS clases.js x
JS clases.js > nombreDeMiClase
1 /*Comenzamos utilizando la palabra reservada "class" */
2 class nombreDeMiClase{
3     /*Una clase cuenta con un método(función) constructor, éste se ejecutará CADA VEZ QUE INSTANCIE UN OBJETO */
4     constructor(parametrosDeCreacion){
5         console.log("Nuevo objeto creado");
6         this.variableInterna=2;
7         /*Cada instancia de la clase contará con variables internas, para poder declararlas y utilizarlas
8         * necesitamos colocar un "this." antes de la variable.
9         */
10    }
11
12    static variableEstatica = 4;
13    /*La palabra "static" es una variable que puede utilizarse SIN NECESITAR UNA INSTANCIA, además, todas sus
14    * instancias pueden acceder a ella de igual manera. SI ALGUNA INSTANCIA CAMBIA LA VARIABLE ESTÁTICA, todas las
15    * instancias se enterarán.
16    */
17    metodo1(){
18        /*Los métodos son funciones que sólo puede utilizar una instancia de la clase.*/
19        console.log("¡Soy un método de la clase!")
20    }
21    metodo2 = () =>{
22        console.log(`Soy una función flecha, puedo incrustar variables: ${this.variableInterna}, todo dentro
23        de una clase. ¡Una locura!
24        `)
25    }
26 }
```

JS clases.js > ...

```
27
28 /*Una vez que terminé de definir mi clase, es hora de instanciar*/
29 /**
30  * Usaremos el operador "new" El cual crea una instancia de la clase.
31  */
32 let instancia = new nombreDeMiClase(); // se ejecutará el constructor diciendo "Nuevo objeto creado" (constructor)
33
34 /*Nota cómo ahora la instancia cuenta con las variables y métodos definidos en la clase previamente */
35 console.log(instancia.variableInterna);
36 instancia.metodo1();
37 instancia.metodo2();
38 /*Para usar la variable estática, no es necesaria la instancia, simplemente lo llamamos desde la clase */
39
40 nombreDeMiClase.variableEstatica;
41
42 /*La magia está en que, como la variable es un molde, se pueden crear múltiples instancias de ésta*/
43
44 let instancia_2 = new nombreDeMiClase(); // Se ejecutará el constructor diciendo "Nuevo objeto creado" (constructor)
45 let instancia_3 = new nombreDeMiClase(); // Se ejecutará el constructor diciendo "Nuevo objeto creado" (constructor)
46
47 /*¡Cada instancia será diferente de las otras en cuanto a sus variables y métodos (excepto los static) */
```



Ejemplo en vivo

- ✓ Se declarará una clase **Persona**, la cual debe crearse con un nombre que identifique la instancia.
- ✓ Además, habrá una variable estática utilizable para todos.
- ✓ Se comprobará la individualidad entre las dos instancias.

JS clases.js x



JS clases.js > Persona > getEspecie

```
48
49 class Persona{
50     constructor(nombre){
51         this.nombre = nombre;
52     }
53     static especie = "humano";
54     saludar = () =>{
55         console.log(`¡Hola, soy ${this.nombre}, mucho gusto!`)
56     }
57     getEspecie = () =>{
58         console.log(`Aunque no lo creas, soy un ${Persona.especie}`)
59     }
60 }
61 let persona1 = new Persona("Jorge");
62 let persona2 = new Persona("Catalina");
63 persona1.saludar();
64 persona2.saludar();
65 persona1.getEspecie();
66 persona2.getEspecie();
```



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

powerShell + - □ ✕

```
¡Hola, soy Jorge, mucho gusto!
¡Hola, soy Catalina, mucho gusto!
Aunque no lo creas, soy un humano
Aunque no lo creas, soy un humano
```



Hands on lab

En esta instancia de la clase **repasaremos cómo se crean las clases.**

¿De qué manera?

El profesor demostrará cómo hacerlo y tú lo puedes ir replicando en tu computadora. Si surgen dudas las puedes compartir para resolverlas en conjunto de la mano de los tutores.

Creación de una clase contador

¿Cómo lo hacemos? **Se creará una clase que permitirá llevar cuentas individuales según cada responsable.**

- ✓ Definir clase Contador
- ✓ La clase se creará con un nombre, representando al responsable del contador.
- ✓ El contador debe inicializarse en 0
- ✓ Debe existir una variable estática que funcione como **contador global** de todas las instancias de contador creadas.

Se espera una duración de 10 minutos.

Creación de una clase contador

- ✓ Definir el método `getResponsable`, el cual debe devolver el responsable de dicho contador.
- ✓ Definir el método `contar`, el cual debe incrementar, tanto su cuenta individual, como la cuenta global.
- ✓ Definir el método `getCuentaIndividual`, el cual debe devolver sólo la cuenta individual del contador
- ✓ Definir el método `getCuentaGlobal`, el cual debe devolver la variable estática con el conteo global.
- ✓ Realizar prueba de individualidad entre las instancias.

¿Preguntas?

Opina y valora
esta clase

Resumen de la clase hoy

- ✓ Repaso sobre tipos de datos y variables en Javascript
- ✓ Javascript y ES6
- ✓ Funciones en Javascript
- ✓ Uso de clases en Javascript

Muchas gracias.

#DemocratizandoLaEducación